

EC8208 – CA02 Viva Cheat-Sheet

MediConnect – Microservices Clinic Appointment Booking Platform

30-Second System Pitch

MediConnect is a **microservices web platform** where patients **book clinic appointments in advance** and attend in person. A **React** app calls an **API Gateway** that routes to independent services (Auth, Appointment, Records, Notification), each with **its own database**, communicating **synchronously via REST** and **asynchronously via RabbitMQ**, deployed as **containers on Kubernetes**.

Answer Framework (use every time)

Driver (requirement/quality attribute) → **Decision** → **Alternative** rejected → **Trade-off** accepted.

“Everything in software architecture is a trade-off.” Always name the downside.

Key Numbers

99.9% uptime • <500 ms (95% of requests) • 10,000 concurrent users • TLS 1.2+ • JWT expiry 2h.

Golden Answers (memorise)

- **Notification service down?** Booking still succeeds; the `appointment.booked` event stays durably in RabbitMQ and is processed on recovery. No booking lost.
- **Single point of failure?** Gateway + broker; mitigated by **replicating the gateway** and **clustering RabbitMQ**.
- **Healthcare needs ACID – why NoSQL?** Polyglot: **PostgreSQL** for booking/payment (ACID), **MongoDB** for flexible records/EHR.
- **Scale to 10k users?** Stateless services + Kubernetes HPA + load-balanced gateway + Redis cache + DB indexes.

Quality Attributes → Tactic

Scalability	Stateless services, K8s HPA, Redis, indexing
Performance	Gateway TLS offload, Redis cache, indexed queries
Security	TLS, encryption at rest, JWT/RBAC, audit logs
Availability	Replicas, self-healing, broker decoupling
Maintainability	Service boundaries, layered code, CI/CD
Reliability	Transactional DB, Saga, idempotency, health checks

Golden Rules for the Viva

- End *every* answer with a **trade-off** – benefits alone is advertising, not justification.
- Tie each choice to a **requirement / quality attribute**.
- Be **honest** about prototype simplifications (all-Node, MongoDB only, Docker Compose instead of K8s). Bluffing gets punished on the follow-up.
- Know your **diagrams** cold: context, component, deployment, class, DFD.

Decisions: Choice → Why → Alternative Rejected → Trade-off

Decision	Why (driver)	Alternative rejected	Trade-off accepted
Microservices	Independent scaling, fault isolation, maintainability, growth to many clinics	Monolith/layered: scales as one block, one failure kills all; client-server: no fault isolation	Operational complexity, distributed data, eventual consistency
API Gateway	Single entry: central auth, TLS, routing, rate limiting; hides services	Clients call services directly: exposed services, duplicated auth, CORS	Gateway is a SPOF → run replicas behind a load balancer
DB-per-service (PostgreSQL + MongoDB)	Loose coupling; each service owns/evolves its data; polyglot fit	Shared database: hidden coupling, schema changes break others	Cross-service queries harder; needs Saga for consistency
RabbitMQ (events)	Decouple notification/records from booking path; resilience	Synchronous calls: booking blocks, a down service cascades failure; Kafka: overkill for this scale	Eventual consistency; a broker to run/monitor
React (SPA)	Responsive, reusable components, large ecosystem, fast dev	Angular (heavier); plain JS (hard to scale UI)	Build step, weak SEO (irrelevant behind login)
Node.js / Express	Non-blocking I/O for I/O-bound APIs; same language as frontend; ecosystem	Heavier frameworks; Spring Boot kept where strong typing/transactions matter	Poor for CPU-heavy work (none here)
JWT + OAuth 2.0	Stateless auth: any replica verifies without a session store; RBAC	Server sessions: need sticky sessions / shared store	Token revocation harder → short expiry
Redis (cache)	Cache hot reads (doctors, slots) to meet <500 ms, cut DB load	No cache: repeated DB hits, slower	Cache invalidation complexity, extra infra
Docker + Kubernetes	Consistent envs, HPA auto-scaling, self-healing, rolling deploys	VMs (heavier); manual deploys (inconsistent)	Kubernetes operational complexity
Saga pattern	Consistency across service DBs via compensating actions	Two-phase commit: blocking, tight coupling, low availability	Must design compensations; eventual consistency